
ACTIVE-IA

Devolución de Trabajo Práctico

Materia:	PROG4A26 - Programacion 4 Agosto 2026
Comisión:	COMI-12 (2026)
Trabajo:	TP Integrador Unidad 3 — POO: de Java a Python
Alumno:	Miguel Angel Pichulman Cari

Calificación Final: 90/100

EVALUACIÓN POR CRITERIOS

C1: Diagnóstico y corrección de java-ismos

18/20



Excelente diagnóstico de los java-ismos en el informe y en el demo de síntomas. El código remueve correctamente los getters preventivos innecesarios, limpia el ruido sintáctico e inicializa los atributos mutables de forma segura. Se restó un mínimo puntaje por no realizar la copia defensiva en el constructor de Polígono y porque un par de justificaciones en el informe no utilizaron las etiquetas conceptuales exactas de las tres inversiones.

DESGLOSE POR SUBCRITERIO

C1.1 6/6 · OK

La tabla en informe.md identifica correctamente los 8 java-ismos de la clave con todas sus columnas.

C1.2 5/6 · OK

Correcciones aplicadas de forma muy efectiva. Se descontó un mínimo puntaje debido a que en Polígono.__init__ se asignan los lados de forma directa sin realizar copia defensiva de entrada.

C1.3 2/3 · WARNING

Se justifican las correcciones, aunque para los java-ismos 6 y 8 se emplearon justificaciones alternativas en lugar de apoyarse estrictamente en las tres inversiones conceptuales definidas en la unidad.

C1.4 3/3 · OK

El archivo demo_sintomas.py demuestra perfectamente dos síntomas con salida reproducible por consola.

C1.5 2/2 · OK

Se identifica correctamente a Lado.longitud como la única propiedad que requería validación activa, evitando la sobre-sintaxis de properties vacías en el resto del modelo.

C2: Modelado de relaciones estructurales

20/25



Muy buen modelado de la agregación Taller-Polígono y la asociación con Etiqueta utilizando dataclasses inmutables. Sin embargo, la relación de composición Polígono-Lado no fue plasmada de forma genuina, ya que se reciben los Lados ya creados desde afuera del constructor y no se realiza validación de cantidad alguna contra `lados_esperados()`. Asimismo, en el informe se omitió la explicación de la asociación.

DESGLOSE POR SUBCRITERIO

C2.1 5/5 · OK

Clase Taller modelada correctamente como agregación; recibe polígonos ya fabricados externos y define inventario y restauración.

C2.2 5/5 · OK

Etiqueta implementada como dataclass congelada y aceptada de forma opcional por Lado.

C2.3 0/3 · ERROR

No se conserva la composición estructural Polígono-Lado. Los lados se instancian externamente y se pasan por parámetro al constructor del polígono sin que este controle su ciclo de vida, omitiendo además la validación contra `lados_esperados()`.

C2.4 5/5 · OK

Se implementa correctamente la copia defensiva de salida mediante conversión a tuplas en `Poligono.lados` y `Taller.inventario`.

C2.5 4/7 · WARNING

Se responde de manera clara sobre composición y agregación en el código, pero se omitió por completo el análisis de la relación de asociación.

C3: Herencia justificada por dominio

18/20



Polígono se declara correctamente como clase abstracta con el decorador `@abstractmethod`, garantizando la falla temprana en runtime. Las subclases implementan los métodos esperados y se define con solidez la decisión de mantener la herencia en `PoligonoRegular`. No obstante, se descuenta puntaje debido a la ausencia total de validación del número de lados recibidos en los constructores.

DESGLOSE POR SUBCRITERIO

C3.1 6/6 · OK

Polígono hereda correctamente de ABC y `lados_esperados` está decorado con `@abstractmethod`, produciendo un `TypeError` en tiempo de instanciación.

C3.2 2/5 · WARNING

Se crean las subclases `Pentagono` y `Hexagono` e implementan sus correspondientes retornos de `lados_esperados()`, pero ninguna de las subclases (ni la clase base) realiza la validación de que la cantidad de lados recibidos coincida con dicha expectativa.

C3.3 9/9 · OK

Se argumenta con solidez semántica de dominio la decisión de sostener a `PoligonoRegular` dentro de la jerarquía de herencia y se refleja de manera congruente en el código.

C4: Contratos: ABC vs Protocol

15/15

La parte de contratos está impecable. Se implementa correctamente la interfaz Exportable como typing.Protocol, logrando integrar la clase de terceros PlanoCAD sin herencia forzada ni modificaciones. El polimorfismo por duck typing funciona de forma nativa en la función exportar_todo.

DESGLOSE POR SUBCRITERIO

C4.1 4/4 · OK

Exportable se declara como Protocol y Poligono cumple con la firma de exportar().

C4.2 4/4 · OK

PlanoCAD se mantiene sin modificaciones e integra estructuralmente el método exportar() sin heredar de las clases del dominio.

C4.3 4/4 · OK

La función exportar_todo recibe la lista tipada, itera y consume el método polimórficamente sin recurrir a comprobaciones de tipo manuales.

C4.4 3/3 · OK

Explicación clara en informe.md fundamentando el uso de Protocol frente a ABC para componentes externos y conectando de manera coherente con el análisis de la Parte 3.

C5: Integradora: código, UML, informe y demo

19/20

Excelente entrega integradora. El script de demostración main.py expone claramente las decisiones de diseño mediante ejecución y captura de errores. El diagrama de clases en Mermaid está sumamente completo y respeta la sintaxis del estándar. Se aplicó un leve descuento por la omisión de la relación de asociación en la sección de relaciones del informe escrito.

DESGLOSE POR SUBCRITERIO

C5.1 4/4 · OK

Módulos de la entrega completos y libre de errores de importación o sintaxis. parte1_diagnostico.py se entrega corregido.

C5.2 5/5 · OK

El archivo modelo_final.md cuenta con un diagrama de clases detallado que utiliza correctamente las notaciones de relaciones UML de Mermaid.

C5.3 5/6 · OK

El informe cumple con la concisión y la estructura solicitada, descontando 1 punto por la omisión del análisis de la relación de asociación en las relaciones de la parte 2.

C5.4 4/4 · OK

main.py funciona de manera robusta y expone activamente la demo de falla temprana, agregación, composición y exportación polimórfica.

C5.5 1/1 · OK

La estructura del proyecto respeta con exactitud el árbol de directorios requerido.

FORTALEZAS

- Excelente diagnóstico y corrección de los java-isms, demostrando los efectos de las mutabilidades compartidas con un robusto `demo_sintomas.py`.
- Correcto y elegante diseño de tipado estructural mediante `typing.Protocol` para resolver el acoplamiento con librerías externas sin herencias forzadas.
- Elaboración de un diagrama de clases UML sumamente claro y prolijo utilizando código Mermaid respetando todas las especificaciones.

RECOMENDACIONES

1. Modificar el constructor de `Polígono` o de sus subclases de manera que los objetos `Lado` sean contruidos de forma interna (composición dura) y no recibiendo ya la lista por parámetros desde el exterior.
2. Incorporar una validación explícita (e.g. `raise ValueError`) en los constructores de los polígonos para asegurar que la cantidad de lados suministrada sea exactamente igual a `lados_esperados()`.
3. Completar en el informe escrito la diferenciación de las tres relaciones estructurales incluyendo el análisis teórico-práctico de la relación de asociación (`Lado -> Etiqueta`).

COMENTARIOS DEL EVALUADOR

El trabajo práctico de Miguel Ángel demuestra una comprensión madura de la transición de Java a Python, logrando limpiar el ruido sintáctico e implementar patrones altamente idiomáticos. La arquitectura de contratos y el diagrama UML final son excelentes, quedando pendiente pulir la realización efectiva de la composición en el código mediante la fabricación interna y validación de lados.